

ASSIGNMENT 9 — MACHINES

COMP 3010 — ORGANIZATION OF PROGRAMMING LANGUAGES

Remember: For every answer (including multiple choice) explain *how* you got your answer and *why* you think it is the correct one. This explanation will be part of your grade.

1. CALL STACKS

Exercise 1. Do *Concepts In Programming Languages* Exercise 7.2 on tail recursion and iteration (page 193).

You may write your answer in any language of your choosing from this list: Standard ML, Ruby, Scheme, Python, or C/C++. No matter what language you choose to write your answer in, make sure you write a *non-recursive* program using a “while” or “for” loop that is equivalent to the function `mult` given in Exercise 7.2 (that is to say, your non-recursive function should always return the same number as the recursively-defined `mult` for any pair of arguments.)

Hint: The tail recursive function `mult` in Exercise 7.2 can be simplified by removing the special case where `a = 1` sharing the constant variable `b` like so:

```
fun mult(a, b) =  
  let fun mult1(a, result) =  
        if a = 0  
        then result  
        else mult1(a-1, result+b)  
      in  
        mult1(a, 0)  
      end;  
end;
```

You may base your answer off of this simplified definition of `mult`.

Exercise 2. Do *Concepts In Programming Languages* Exercise 7.8 on static and dynamic scope (pages 196-197).

Extra Credit Exercise 1. Do *Concepts In Programming Languages* Exercise 7.11 on λ -calculus and scope (page 198).

Exercise 3. Do *Concepts In Programming Languages* Exercise 7.12 on function calls and memory management (pages 198-199).

Extra Credit Exercise 2. Do *Concepts In Programming Languages* Exercise 7.13 on function returns and memory management (pages 199-200).

2. ABSTRACT MACHINES

Recall this definition of an abstract machine for calculating the answers to arithmetic problems:

$$n ::= 0 \mid 1 \mid 2 \mid 3 \mid \dots$$

$$A ::= \underline{n} \mid \text{plus}(A, A) \mid \text{minus}(A, A) \mid \text{times}(A, A)$$

$$K ::= \text{done} \mid F; K$$

$$F ::= \text{plus}(\square, A) \mid \text{plus}(n, \square) \mid \text{minus}(\square, A) \mid \text{minus}(n, \square) \mid \text{times}(\square, A) \mid \text{times}(n, \square)$$

$$S ::= \text{Eval}\langle A \mid K \rangle \mid \text{Ret}\langle n \mid K \rangle$$

$$\begin{aligned} \text{Eval}\langle \text{plus}(A_1, A_2) \mid K \rangle &\mapsto \text{Eval}\langle A_1 \mid \text{plus}(\square, A_2); K \rangle \\ \text{Eval}\langle \text{minus}(A_1, A_2) \mid K \rangle &\mapsto \text{Eval}\langle A_1 \mid \text{minus}(\square, A_2); K \rangle \\ \text{Eval}\langle \text{times}(A_1, A_2) \mid K \rangle &\mapsto \text{Eval}\langle A_1 \mid \text{times}(\square, A_2); K \rangle \\ \text{Eval}\langle \underline{n} \mid K \rangle &\mapsto \text{Ret}\langle n \mid K \rangle \end{aligned}$$

$$\begin{aligned} \text{Ret}\langle n \mid \text{plus}(\square, A); K \rangle &\mapsto \text{Eval}\langle A \mid \text{plus}(n, \square); K \rangle \\ \text{Ret}\langle n \mid \text{plus}(n', \square); K \rangle &\mapsto \text{Ret}\langle n' + n \mid K \rangle \\ \text{Ret}\langle n \mid \text{minus}(\square, A); K \rangle &\mapsto \text{Eval}\langle A \mid \text{minus}(n, \square); K \rangle \\ \text{Ret}\langle n \mid \text{minus}(n', \square); K \rangle &\mapsto \text{Ret}\langle n' - n \mid K \rangle \quad (\text{if } n' \geq n) \\ \text{Ret}\langle n \mid \text{times}(\square, A); K \rangle &\mapsto \text{Eval}\langle A \mid \text{times}(n, \square); K \rangle \\ \text{Ret}\langle n \mid \text{times}(n', \square); K \rangle &\mapsto \text{Ret}\langle n' \times n \mid K \rangle \end{aligned}$$

Exercise 4. Using the above rules for executing programs in the abstract machine, show the step-by-step calculating the final state of the machine command $\text{Eval}\langle \text{times}(\underline{2}, \text{minus}(\text{plus}(\underline{3}, \underline{4}), \underline{5})) \mid \text{done} \rangle$.

Hint: Beginning with the starting machine state (S_0) above, keep applying the machine steps listed above on at a time to transition between states

$$\begin{aligned} S_0 &\mapsto S_1 \\ &\mapsto S_2 \\ &\mapsto \dots \\ &\mapsto S_{\text{done}} \end{aligned}$$

until you reach a final state S_{done} that should look like either $\text{Ret}\langle n \mid \text{done} \rangle$ or $\text{Ret}\langle b \mid \text{done} \rangle$. Each machine state has *at most one* transition rule that could possibly apply, so you will never have to choose between multiple options.

Recall the Krivine machine for calculating answers to λ -calculus programs (extended with numbers $n ::= 0 \mid 1 \mid 2 \mid \dots$):

$$\begin{aligned} M &::= n \mid x \mid M \ M \mid \lambda x. M \\ \Gamma &::= \varepsilon \mid x = [\Gamma]M, \Gamma \\ \Pi &::= \text{done} \mid [\Gamma]M; \Pi \end{aligned}$$

$$\begin{aligned}
 \langle M \ M' \mid \Gamma \mid \Pi \rangle &\mapsto \langle M \mid \Gamma \mid [\Gamma]M; \Pi \rangle \\
 \langle \lambda x.M \mid \Gamma \mid [\Gamma']M'; \Pi \rangle &\mapsto \langle M \mid x = [\Gamma']M', \Gamma \mid \Pi \rangle \\
 \langle x \mid \Gamma \mid \Pi \rangle &\mapsto \langle M' \mid \Gamma' \mid \Pi \rangle \quad (\text{if } (x = [\Gamma']M') \in \Gamma)
 \end{aligned}$$

Exercise 5. (a) Show the step-by-step β -reduction in the small-step semantics of the λ -calculus to simplify the expression

$$(\lambda x.(\lambda y.(\lambda x.y \ 30) \ 20) \ (\lambda z.x)) \ 10$$

down to a final answer of the form of a number n or function $\lambda x'.M'$ with some parameter x' and some body M' .

(b) Using the above rules of the Krivine machine and starting from the initial machine state of

$$\langle (\lambda x.(\lambda y.(\lambda x.y \ 30) \ 20) \ (\lambda z.x)) \ 10 \mid \varepsilon \mid \text{done} \rangle$$

show the step-by-step transition sequence for calculating the final state of the machine, which should have the form of either $\langle \lambda z.M \mid \Gamma \mid \text{done} \rangle$ or $\langle n \mid \Gamma \mid \text{done} \rangle$ for some environment Γ .

(c) Compare the answer you get to both previous steps. Are they the same? Should they be the same? Why or why not?

Recall that the local names in an expression can be eliminated by translating to de Bruijn indexes like so:

$$\begin{aligned}
 i &::= 0 \mid 1 \mid 2 \mid \dots \\
 M &::= \text{var}_i \mid M \ M \mid \lambda M
 \end{aligned}$$

$$\begin{aligned}
 \llbracket x \rrbracket &= x \\
 \llbracket M \ M' \rrbracket &= \llbracket M \rrbracket \ \llbracket M' \rrbracket \\
 \llbracket \lambda x.M \rrbracket &= \lambda([0/x] \llbracket M \rrbracket)
 \end{aligned}$$

$$\begin{aligned}
 [i/x]x &= \text{var}_i \\
 [i/x]y &= y & (\text{if } x \neq y) \\
 [i/x](M \ M') &= ([i/x]M) \ ([i/x]M') \\
 [i/x]\lambda M &= \lambda([i+1/x]M)
 \end{aligned}$$

These nameless expression can be run inside of a de Bruijn machine that only uses numeric positional offsets, rather than looking up variables by name:

$$\begin{aligned}
 \Gamma &::= \varepsilon \mid [\Gamma]M, \Gamma \\
 \Pi &::= \text{done} \mid [\Gamma]M; \Pi
 \end{aligned}$$

$$\begin{aligned}
 \langle M \ M' \mid \Gamma \mid \Pi \rangle &\mapsto \langle M \mid \Gamma \mid [\Gamma]M'; \Pi \rangle \\
 \langle \lambda M \mid \Gamma \mid [\Gamma']M'; \Pi \rangle &\mapsto \langle M \mid [\Gamma']M', \Gamma \mid \Pi \rangle \\
 \langle \text{var}_i \mid \Gamma \mid \Pi \rangle &\mapsto \langle M' \mid \Gamma' \mid \Pi \rangle \quad (\text{if } \Gamma \text{ at index } i = [\Gamma']M')
 \end{aligned}$$

Notice how in this machine, both the environment Γ and the stack Π are just plain arrays/lists of closures $[\Gamma]M$.

Extra Credit Exercise 3.

- (a) Draw a syntax tree of the λ -calculus expression $(\lambda x.(\lambda y.(\lambda x.y\ 30)\ 20)\ (\lambda z.x))\ 10$. Then, draw an arrow from each leaf which references a variable to the λ that introduces the variable it refers to. In other words, walk up the tree from the reference leaf containing a variable like x and stop at the first λ you encounter which introduces that variable (x) as a parameter; this is the parameter the leaf refers to.

Next to each variable reference leaf, write down the number of λ s that were *skipped* between each reference to a variable and the λ -parameter it points to. For example, if a variable reference leaf points to the nearest λ up the tree (even if there are other nodes like application in the middle), write a 0 next to it. If there is only one other λ node between a variable leaf and the λ introducing a parameter of the same name as the variable, write a 1. And so on.

- (b) Show how to translate the λ -calculus expression $(\lambda x.(\lambda y.(\lambda x.y\ 30)\ 20)\ (\lambda z.x))\ 10$ to its nameless de Bruijn form by following the definition of

$$\llbracket (\lambda x.(\lambda y.(\lambda x.y\ 30)\ 20)\ (\lambda z.x))\ 10 \rrbracket$$

Note, in the base case where you get to a number n (in this case, $n = 10$ or 20 or 30), assume that $\llbracket n \rrbracket = n$.

Hint: The index i you get at each var_i in the translated expression should correspond to the number of skipped λ s you found in the syntax tree from part (a). In other words, if you take your syntax tree from part (a) and erase all the variable names, you should get a new nameless syntax tree that corresponds to your answer to this question.

- (c) Given that your answer to part (b) is the term M which no longer has any variables on it, use the above rules for the de Bruijn machine and start from the initial machine state $\langle M \mid \varepsilon \mid \text{done} \rangle$ to show the step-by-step transition sequence for calculating the final state of the machine, which should have the form of either $\langle \lambda M \mid \Gamma \mid \text{done} \rangle$ or $\langle n \mid \Gamma \mid \text{done} \rangle$ for some environment array Γ .